

NYU, Tandon School of Engineering

Bridge to CS

Homework #15

Submission Instructions

1. For this assignment, you should turn in 1 `.cpp` file:
 - Name your file: `hw15.cpp`
 - Your program should include a `main()` function.
 2. Pay special attention to the style of your code. Indent your code correctly, choose meaningful names for your variables, define constants where needed, choose the most appropriate control flow statements, break down your solutions by defining functions, etc.
 3. **Important:** You must implement your own linked list and node classes for this assignment. Do not use `std::list` or other STL containers for storing the input data.
-

Question 1: Expense Splitting with Linked Lists

Imagine a group of friends went out to dinner together. When the bill comes, they all give different amounts to contribute to the total, but afterwards they wish to balance everything out equally so that each person pays the same amount. Knowing how much each person is supposed to pay is simple enough—just add up all the amounts paid and divide by the number of people—but that doesn't tell anyone *how* to go about distributing the payments: who should give what amount to whom?

Your Task:

Write a program that will print out a step-by-step list of instructions on how to reimburse everyone, given the name of each person and the amount of money they paid.

Program Requirements:

1. **File Input:** The program should prompt the user to enter a file name. If the file doesn't exist, prompt again until a valid file is provided.
2. **Linked List Storage:** All information from the external file must be read into a linked list that you implement yourself. You must design your own linked list and node classes.
3. **Node Design:** The node class should represent a person. Suggested attributes:
 - Name (string)
 - Amount paid (double) — from the input file

- Amount owed (double) — can be initialized to 0 and calculated later
4. **Name Handling:** You must be able to read full names, accounting for both 2-part names (e.g., “John Smith”) and 3-part names (e.g., “Jason J. Jones”).
 5. **Output Format:** Print reimbursement instructions showing who gives money to whom, and the final target amount everyone should have paid.

Input File Format:

Each line contains an amount paid followed by a space and then the person’s full name:

`<amount> <full name>`

Amounts and names are separated by a single space (not a tab).

Example:

Input File (data.txt):

```
76.8 Jason J. Jones
89 Natalie Johnson
11 Dalton Smith
44.2 Sheryl Carter
0 John Martin
```

```
Enter the filename: data.txt
Sheryl Carter, you don't need to do anything
Dalton Smith, you give Jason J. Jones $32.60
Dalton Smith, you give Natalie Johnson $0.60
John Martin, you give Natalie Johnson $44.20
In the end, you should all have spent around $44.20
```

Explanation:

Total amount paid: $76.8 + 89 + 11 + 44.2 + 0 = 221$

Target per person: $221 \div 5 = 44.20$

- Jason J. Jones paid \$76.80 (overpaid by \$32.60)
- Natalie Johnson paid \$89.00 (overpaid by \$44.80)
- Dalton Smith paid \$11.00 (underpaid by \$33.20)
- Sheryl Carter paid \$44.20 (exactly right)
- John Martin paid \$0.00 (underpaid by \$44.20)

People who underpaid need to give money to people who overpaid until everyone is balanced.

Additional Notes:

- After reading data into a linked list, you may transfer it to another data structure (e.g., vector) for processing if you find that more convenient.
- Don’t worry about rounding to exactly two decimal places for dollar amounts.

- You can assume the data file will have at least 2 entries with valid names and amounts; we will not test with empty files or badly formatted data.
- The program should not modify the input file—you are only reading from it.
- When prompting for a file that doesn't exist, display an error message and prompt again.

Hint:

A good approach is to:

1. Calculate the target value (total sum $\div$ number of people)
2. For each person, calculate how much they owe or are owed (amount paid $-$ target)
3. Positive balance means they overpaid (are owed money)
4. Negative balance means they underpaid (owe money)
5. Match people who owe money with people who are owed money